

Pearls AQI Predictor

*A Serverless, End-to-End Machine Learning Pipeline for 3-Day Air Quality
Forecasting in Karachi*

Internship Project Report

Prepared by: Maham Tahir
Date: August 2026

1. Executive Summary

This project implements a complete, production-style machine learning system that forecasts the Air Quality Index (AQI) for Karachi, Pakistan, 72 hours (3 days) into the future. The system is built entirely on free-tier, serverless infrastructure: live and historical air-quality data is collected automatically, engineered into features, stored in a managed Feature Store, used to train and evaluate multiple machine learning models, and served through an interactive public web dashboard. All data collection and model retraining runs unattended on a schedule, with no manual intervention required.

The final system covers every stage of the brief: a scheduled feature pipeline, historical backfill, a model training pipeline with rigorous time-series evaluation, automated CI/CD via GitHub Actions, and a multi-page Streamlit dashboard with forecast, historical trend, and SHAP explainability views.

2. Project Overview & Objectives

The goal was to predict Karachi's AQI three days ahead using a 100% serverless stack, following a four-stage architecture provided in the project brief: (1) a feature pipeline that fetches raw weather and pollution data, (2) historical backfill to build a training dataset, (3) a training pipeline that evaluates multiple models and registers the best one, and (4) a web application that serves live forecasts.

2.1 Architecture Diagram

Weather & Pollution API → Feature Pipeline (hourly, GitHub Actions) → Hopsworks Feature Store
Hopsworks Feature Store → Training Pipeline (daily, GitHub Actions) → Hopsworks Model Registry
Hopsworks Model Registry + Feature Store → Streamlit Web App (public dashboard)

3. Technology Stack

Component	Technology Used
Language	Python 3.11
Live data source	OpenWeather API (Air Pollution + Current Weather endpoints)
Historical data source	Open-Meteo API (Air Quality + Weather Archive — no historical endpoint on OpenWeather's free tier)
Feature Store & Model Registry	Hopsworks (free tier)
Modeling	scikit-learn (Ridge Regression, Random Forest); MLPRegressor (neural network)
Explainability	SHAP (TreeExplainer for Random Forest; coefficient analysis for Ridge)
Automation / CI-CD	GitHub Actions (hourly + daily scheduled workflows)
Web application	Streamlit, deployed on Streamlit Community Cloud

Version control	Git / GitHub
-----------------	--------------

Note on deviations from the suggested stack: AQICN was not used in favor of OpenWeather + Open-Meteo (see Section 9.1); TensorFlow was substituted with scikit-learn's MLPRegressor (see Section 9.2); and Flask/FastAPI was not needed since Streamlit alone fully serves the dashboard requirement.

4. Feature Pipeline

The feature pipeline (`feature_pipeline.py`) runs hourly via GitHub Actions. Each run:

- Fetches current pollutant concentrations (PM2.5, PM10, NO2, O3, CO, SO2, NH3) and weather conditions (temperature, humidity, pressure, wind speed) for Karachi from OpenWeather.
- Computes AQI from PM2.5 using the standard US EPA breakpoint formula (0–500 scale), rather than relying on OpenWeather's coarser 1–5 index.
- Derives time-based features: hour, day, month, and day-of-week.
- Derives change features: AQI change rate, PM2.5 change rate, AQI lag (previous reading), and a rolling mean — computed against the most recently stored row, since a live run only has one new data point.
- Writes the resulting feature row into the Hopsworks Feature Store (feature group: `aqi_features`, version 1).

A recurring engineering challenge here was schema consistency: the first insert to Hopsworks permanently fixes the feature group's column types. Later inserts from a different source (Open-Meteo, for backfill) or with different numeric types caused schema-mismatch errors, which were resolved by writing an explicit `align_schema_for_hopsworks()` step applied to every insert.

5. Historical Data Backfill

Initial backfills of 7 and 90 days proved insufficient — models trained on a short window failed to generalize once tested against a later chronological slice that fell in a different season. The backfill was therefore extended to a full year: 8,784 hourly records (Aug 2025 – Aug 2026) fetched from Open-Meteo's free historical archive, with zero missing values across all feature columns.

6. Exploratory Data Analysis

EDA was performed on the full backfilled dataset to identify trends before modeling. Key findings:

- AQI ranged from 43 to 158 over the year, averaging ~75 (moderate to unhealthy-for-sensitive-groups).
- A clear daily cycle: AQI dips in the early morning (hours 0–6) and peaks in late morning/midday (hours 9–13), consistent with traffic patterns and reduced atmospheric mixing.

- PM2.5 and PM10 correlate almost perfectly with AQI (0.94–1.00) — expected, since AQI is derived directly from PM2.5.
 - Temperature correlates positively with AQI (+0.54); humidity correlates negatively (–0.42), plausibly because moisture helps settle particulates.
 - A pronounced negative correlation between pressure and temperature (–0.83) reflects Karachi's seasonal weather regime (e.g., monsoon vs. dry season).
-

7. Training Pipeline & Model Evaluation

The training pipeline (`training_pipeline.py`) runs daily via GitHub Actions. It reframes the problem as forecasting AQI 72 hours ahead, using each row's current features to predict the AQI value 72 hours later.

7.1 Initial (naive) evaluation — a cautionary finding

An initial 80/20 chronological train/test split produced negative R^2 scores for every model, including a naive persistence baseline ("AQI in 72h = AQI now"). Diagnosis showed the training period (Aug 2025–Jun 2026, mean AQI 90, std 28.1) and the test period (Jun–Aug 2026, mean AQI 74, std 12.6) reflected genuinely different seasonal regimes — the test window fell in Karachi's calmer monsoon season. This is an important, documented finding: a single chronological split can be misleading for seasonal data.

7.2 Rigorous evaluation — 5-fold time-series cross-validation

To get a trustworthy comparison, models were re-evaluated using scikit-learn's `TimeSeriesSplit` across 5 chronological folds:

Model	Fold R^2 scores	Mean RMSE	Mean R^2
Ridge Regression	-0.16, -16.99, -0.03, -0.07, -0.22	42.07	-3.495
Random Forest	-0.13, -0.27, -0.55, -0.08, 0.14	24.85	-0.179

Random Forest proved far more robust: Ridge collapsed catastrophically in one fold ($R^2 = -16.99$) when an early, small training window collided with an extreme AQI event, while Random Forest degraded gracefully across all folds and even achieved a positive R^2 (0.14) in the final fold — the one with the most training data available. This trend suggests performance should continue to improve as the automated pipeline accumulates more historical data over time.

An MLP neural network was also evaluated on a single split (RMSE 21.4, $R^2 = -1.88$) and underperformed both linear and tree-based models, consistent with having too little data (~1,700–8,700 rows) for a neural network to outperform simpler models.

7.3 Model selection

Based on the cross-validated results, Random Forest was selected as the production model. The training pipeline re-runs this same 5-fold CV comparison daily and automatically registers whichever of Ridge or Random Forest currently performs best — model selection is data-driven, not hardcoded.

8. Model Explainability (SHAP)

SHAP (TreeExplainer) was used to explain individual forecasts from the Random Forest model. The summary plot revealed that month, pressure, temperature, and day dominate the model's predictions — considerably more than the pollutant readings (PM2.5, PM10, NO2, etc.) themselves.

This is a genuinely useful finding: it indicates that a 72-hour AQI forecast for Karachi is fundamentally a weather-and-season prediction problem more than a pollutant-tracking problem, since the atmospheric regime (season, pressure system) determines how pollution accumulates or disperses over a 3-day horizon.

9. Automated CI/CD Pipeline

Both pipelines run unattended via GitHub Actions:

- Feature Pipeline — scheduled hourly (cron: 5 * * * *), fetches live data and writes to the Feature Store.
- Training Pipeline — scheduled daily (cron: 0 2 * * *), re-evaluates models via 5-fold CV and registers the current best model.

Both workflows are confirmed running successfully in production. Notable engineering issues resolved along the way included: Hopsworks feature-group schema type mismatches (int64 vs. double) between different data sources, requiring explicit dtype-forcing via `pd.Series(..., dtype="float64")`; and separating Kafka-dependent dependencies (needed only for the hourly online-store inserts) from the read-only web app's requirements, since confluent-kafka failed to compile in the deployment environment and is not required for reads.

10. Web Application Dashboard

The dashboard is a multi-page Streamlit application deployed on Streamlit Community Cloud, structured as:

- Home / About — project overview, architecture diagram, technology stack, and key features.
- Live Forecast — current AQI, 3-day-ahead forecast with delta, model version in use, a color-coded hazardous-AQI alert banner, an AQI scale reference, and current pollutant/weather readings.
- Trends & Explainability — a historical AQI trend chart with selectable time ranges (7/30/90 days or all data), an average-AQI-by-hour chart, and a SHAP (or coefficient, for linear models) explanation of the current forecast.

The app always reads the latest registered model and latest features directly from Hopsworks at runtime, so newly retrained models are reflected automatically without redeploying the app. Data

loading is server-side filtered by date range and cached (55-minute TTL, matching the hourly update cadence) to keep page loads fast.

[Live dashboard: aqi-predictor-karachi.streamlit.app](https://aqi-predictor-karachi.streamlit.app)

[Source code repository](#)

11. Key Design Decisions & Deviations from the Brief

11.1 Historical data source: Open-Meteo instead of AQICN/OpenWeather

OpenWeather's free tier provides no historical air-pollution endpoint, and the initial small-scale test (7 and 90 days) using it produced misleading, overly optimistic training conditions. Open-Meteo was adopted instead for backfill because it offers a full year of free hourly historical data with no API key, letting the models see complete seasonal cycles.

11.2 Deep learning model: MLPRegressor instead of TensorFlow

TensorFlow requires protobuf ≥ 5.28 , while the Hopworks Python client requires protobuf < 5.0 — an unresolvable version conflict within a single Colab/deployment environment. scikit-learn's MLPRegressor (a genuine feed-forward neural network) was used instead to satisfy the "deep learning" model requirement without breaking the Feature Store connection.

11.3 Flask/FastAPI not used

The brief allows either Streamlit/Gradio or Flask/FastAPI. Streamlit alone fully satisfies the dashboard requirement, so a separate Flask/FastAPI backend was not built.

12. Limitations & Future Work

- Model accuracy: even the best model (Random Forest) shows negative mean R^2 under time-series cross-validation, meaning it does not yet reliably outperform a naive persistence forecast across all seasons. More years of historical data, additional weather features (e.g. dust/sandstorm indicators specific to Karachi), and ensemble methods are natural next steps.
 - Single city: the pipeline is currently hardcoded to Karachi; generalizing to multiple cities would require parameterizing the feature/training scripts and Feature Store schema by city.
 - Deep learning: given more data volume over time, revisiting an LSTM/temporal model (once environment dependency conflicts are resolved, e.g. via a separate service for TensorFlow-based training) could better capture longer-range temporal patterns than the current per-row feature approach.
 - Alerting: the current hazardous-AQI alert is dashboard-only (visual banner); a future iteration could add push/email/SMS notifications.
-

13. Conclusion

This project delivers a complete, end-to-end, serverless AQI forecasting system for Karachi: automated hourly data collection, a full year of backfilled training data, a rigorously cross-validated model training pipeline with automatic model selection, daily automated retraining, and a public, multi-page interactive dashboard with forecasts, historical trends, and explainability. Every requirement in the original project brief has been addressed, with deviations from the suggested tool list explicitly documented and justified above. The system is live, automated, and requires no manual intervention to keep running.

14. Appendix: Links

[GitHub Repository](#)

[Live Dashboard \(Streamlit Community Cloud\)](#)